

Compilador Educativo — Análisis Semántico

Estructura del proyecto

```
demo/
├── src/main/antlr4/com/compilador/
│   └── MiLenguaje.g4
├── src/main/java/com/compilador/
│   ├── App.java
│   ├── MiLenguajeLexer.java
│   ├── MiLenguajeParser.java
│   ├── MiLenguajeVisitor.java
│   ├── MiLenguajeBaseVisitor.java
│   └── semantico/
│       ├── SemanticAnalyzer.java
│       ├── SymbolTable.java
│       ├── Scope.java
│       └── Symbol.java
├── parámetro
│   ├── TypeSystem.java
│   └── SemanticError.java
├── ejemplo.txt
└── pom.xml
```

← gramática: define léxico + sintaxis

← orquesta las 3 fases

← generado por ANTLR (no editar)

← generado por ANTLR (no editar)

← generado por ANTLR (no editar)

← generado por ANTLR (no editar)

← visitor que hace el análisis semántico

← gestiona la pila de scopes

← un ámbito (mapa nombre → símbolo)

← un símbolo: variable, función o

← reglas de compatibilidad de tipos

← un diagnóstico con severidad

← programa de prueba

Cómo compilar y ejecutar

```
cd demo
mvn clean package
java -jar target/demo-1.0-jar-with-dependencies.jar ejemplo.txt
```

1. Las tres fases del compilador

Este proyecto implementa las tres primeras fases de un compilador:



Lexer

Parser

SemanticAnalyzer

- **Fase 1 — Léxica:** convierte el texto en tokens (`int`, `x`, `=`, `10`, `;`...).
- **Fase 2 — Sintáctica:** verifica que los tokens estén en el orden correcto y construye el árbol de parseo.
- **Fase 3 — Semántica:** recorre el árbol y verifica que el programa tenga *sentido*: tipos correctos, variables declaradas, funciones existentes, etc.

La fase semántica **no verifica estructura** (eso ya lo hizo el parser). Verifica *significado*.

2. Lo que ANTLR genera automáticamente

La gramática `MiLenguaje.g4` es el punto de partida. A partir de ella, ANTLR genera cuatro archivos Java que **no hay que editar**:

`MiLenguajeLexer.java`

Convierte el texto del archivo fuente en una secuencia de tokens. Cada regla del lexer en la gramática se convierte en un token:

```
INT      : 'int';
ID       : (LETRA | '_' ) (LETRA | DIGITO | '_' )*;
INTEGER  : DIGITO+;
```

`MiLenguajeParser.java`

Contiene los métodos para cada regla del parser. Lo más importante: genera una **clase Context** por cada regla. Por ejemplo, la regla:

```
declaracion : tipo ID (IGUAL expresion)? PYC ;
```

genera `DeclaracionContext`, que tiene métodos para acceder a sus partes:

```
ctx.tipo()      // el nodo "tipo" (int, float, bool...)
ctx.ID()         // el token identificador
ctx.expresion()  // la expresión inicial (puede ser null si no hay)
```

Cada Context es un **nodo del árbol de parseo**.

`MiLenguajeVisitor.java`

Una interfaz con un método `visit...()` por cada regla del parser:

```
public interface MiLenguajeVisitor<T> {
    T visitPrograma(ProgramaContext ctx);
    T visitDeclaracion(DeclaracionContext ctx);
    T visitExprAditiva(ExprAditivaContext ctx);
    // ... uno por cada regla
}
```

MiLenguajeBaseVisitor.java

Implementación por defecto de la interfaz anterior. Cada método simplemente llama a `visitChildren(ctx)`, que visita los nodos hijos y retorna el resultado del último. Es la clase que se **extiende** para escribir el analizador semántico:

```
// SemanticAnalyzer.java
public class SemanticAnalyzer extends MiLenguajeBaseVisitor<String> {
    // sobrescribe solo los métodos que le interesan
}
```

El tipo genérico `String` es lo que devuelven los métodos visit. Se usa para propagar el **tipo** de cada expresión (`"int"`, `"bool"`, `"double"`...).

3. La tabla de símbolos

La tabla de símbolos guarda registro de cada nombre declarado en el programa: qué tipo tiene, si fue inicializado, y en qué ámbito vive. Se implementa en tres clases.

Symbol — un símbolo individual

`Symbol.java` representa una sola entrada en la tabla:

```
Symbol {
    String    nombre        // "x", "suma", "a"
    String    tipo          // "int", "bool", "double"...
    Categoria categoria     // VARIABLE | FUNCION | PARAMETRO
    boolean   inicializado  // ¿tiene valor asignado?
    int       linea         // posición en el fuente (para errores)
    int       columna
}
```

Se crea con factory methods según la categoría:

```
Symbol.variable("x",    "int", true, 1, 4) // int x = 10;
Symbol.funcion ("suma", "int",      4, 4) // int suma(...) siempre
inicializado
```

```
Symbol.parametro("a", "int", 4, 13) // parámetro siempre
inicializado
```

Scope — un ámbito de nombres

`Scope.java` es un mapa `nombre → Symbol` con un puntero a su scope padre:

```
Scope {
    String          nombre    // "global", "funcion_suma_1", "bloque_2"
    Scope           padre     // null si es el scope raíz
    LinkedHashMap<String, Symbol> simbolos
}
```

Tiene tres operaciones clave:

`definir(Symbol)` — agrega el símbolo al mapa local. Si ya existe con ese nombre, retorna `false` (redeclaración).

`resolver(String nombre)` — busca el nombre en el mapa local. Si no lo encuentra, sube al padre y busca allí. Así funciona la visibilidad de variables externas:

```
// Scope.java:25-29
public Symbol resolver(String nombre) {
    Symbol s = simbolos.get(nombre);
    if (s != null) return s;
    if (padre != null) return padre.resolver(nombre); // sube un nivel
    return null; // no existe en ningún scope
}
```

`estaDefinidoLocalmente(String)` — solo busca en el mapa local, sin subir. Se usa para detectar redeclaraciones.

SymbolTable — la pila de scopes

`SymbolTable.java` gestiona una pila de scopes activos:

```
SymbolTable {
    Scope          scopeActual    // siempre apunta al scope más interno
    int            contadorScopes // para nombrar scopes únicamente
    List<Scope>    historial      // todos los scopes creados (para imprimir la
    tabla)
}
```

Al construirse crea automáticamente el scope `"global"`:

```
// SymbolTable.java:17-23
public SymbolTable() {
    Scope global = new Scope("global", null);
    this.scopeActual = global;
    this.historial = new ArrayList<>();
    this.historial.add(global);
}
```

Las operaciones principales:

Método	Qué hace
<code>entrarScope("funcion_suma")</code>	Crea <code>Scope("funcion_suma_1", padre=actual)</code> , lo activa
<code>salirScope()</code>	Vuelve al scope padre
<code>definir(Symbol)</code>	Agrega al scope actual
<code>resolver(String)</code>	Busca desde el scope actual hacia arriba
<code>estaDeclaradoLocalmente(String)</code>	Solo busca en el scope actual

Ciclo de vida de un scope:

```
entrarScope("bloque")      ← se crea al encontrar '{'
    definir(Symbol "local") ← variables declaradas adentro
    resolver("x")           ← busca local; si no, sube al padre
salirScope()               ← se destruye al encontrar '}'
// "local" ya no existe
resolver("local")          ← retorna null → error: no declarada
```

4. El patrón Visitor

Qué es

El patrón Visitor separa el recorrido de una estructura de datos de las operaciones que se hacen sobre ella. En este caso:

- La **estructura** es el árbol de parseo (construido por ANTLR).
- La **operación** es el análisis semántico.

`SemanticAnalyzer` extiende `MiLenguajeBaseVisitor<String>` y sobrescribe solo los métodos que necesita:

```
// SemanticAnalyzer.java:11
public class SemanticAnalyzer extends MiLenguajeBaseVisitor<String> {
    // ...
    @Override
```

```

    public String visitDeclaracion(DeclaracionContext ctx) { ... }

    @Override
    public String visitExprAditiva(ExprAditivaContext ctx) { ... }
    // ...
}

```

Cómo funciona `visit()`

Cuando se llama a `visit(nodo)`, ANTLR despacha al método correcto según el tipo del nodo:

```

visit(nodo DeclaracionContext) → visitDeclaracion(ctx)
visit(nodo ExprAditivaContext) → visitExprAditiva(ctx)
visit(nodo ExprEnteroContext) → visitExprEntero(ctx)

```

Esto permite recorrer el árbol de forma recursiva: un método puede llamar a `visit(hijo)` para obtener el tipo de una subexpresión.

Qué devuelve cada método

Hay una convención en todo el analizador:

Tipo de nodo	Devuelve	Ejemplo
Sentencia (<code>declaracion</code> , <code>if</code> , <code>while</code> ...)	<code>null</code>	Las sentencias no tienen tipo
Expresión (<code>exprAditiva</code> , <code>exprIdentificador</code> ...)	<code>String</code> con el tipo	<code>"int"</code> , <code>"bool"</code> , <code>"double"</code>

Esto permite que las sentencias consulten el tipo de sus expresiones:

```

// visitDeclaracion necesita saber el tipo de la expresión inicial
String tipoExpr = visit(ctx.expresion()); // → "int", "string", etc.

```

5. Sistema de tipos

`TypeSystem.java` centraliza **todas** las decisiones de tipos. Ninguna otra clase toma estas decisiones directamente.

Tipos disponibles

```

TypeSystem.INT      = "int"
TypeSystem.FLOAT    = "float"
TypeSystem.DOUBLE   = "double"
TypeSystem.CHAR     = "char"
TypeSystem.STRING   = "string"
TypeSystem.BOOL     = "bool"

```

```

TypeSystem.VOID    = "void"
TypeSystem.ERROR   = "ERROR"    // centinela interno

```

Reglas de compatibilidad de asignación

TypeSystem.java:14 — esCompatibleAsignacion(hacia, desde):

Variable (hacia)	Acepta (desde)
int	int
float	int, float, double
double	int, float, double
char	char
string	string
bool	bool

Inferencia en expresiones

Método	Operador	Regla	Resultado
inferirAritmetico	+ - * / %	Ambos deben ser numéricos	Tipo más preciso (int < float < double)
inferirRelacional	< > <= >=	Ambos numéricos	bool
inferirIgualdad	== !=	Mismo tipo o ambos numéricos	bool
inferirLogico	&&	Ambos bool	bool
inferirNot	!	bool	bool
inferirNegativo	- (unario)	Numérico	Mismo tipo

El tipo ERROR

Cuando una expresión ya produjo un error, el tipo "ERROR" se propaga sin generar errores adicionales. Así se evita la cascada de mensajes:

```

string texto = "hola";
int suma = texto + 5;    // ← ERROR: string + int no es válido
cout << suma + 1;       // suma es ERROR → no genera segundo error

```

6. Errores y advertencias

`SemanticError.java` representa un diagnóstico con severidad:

```
SemanticError {
    int    linea
    int    columna
    String mensaje
    Severidad severidad // ERROR | ADVERTENCIA
}
```

En `SemanticAnalyzer.java` hay dos métodos privados:

```
// SemanticAnalyzer.java:24
private void error(Token token, String mensaje) {
    errores.add(new SemanticError(..., Severidad.ERROR));
}

// SemanticAnalyzer.java:33
private void warning(Token token, String mensaje) {
    advertencias.add(new SemanticError(..., Severidad.ADVERTENCIA));
}
```

La diferencia es importante:

- Un **error** impide que el programa sea válido → `Compilacion finalizada con errores semanticos.`
- Una **advertencia** es sospechoso pero no fatal → `Compilacion exitosa con advertencias.`

Actualmente el único caso que genera advertencia es usar una variable no inicializada:

```
// SemanticAnalyzer.java:364
if (!simbolo.isInicializado()) {
    warning(token, "variable '" + nombre + "' podría no estar inicializada.");
}
```

El analizador **no se detiene** al encontrar un error. Acumula todos los errores y los muestra al final, para que el programador vea todos los problemas de una sola vez.

7. Recorrido del árbol — paso a paso con `ejemplo.txt`

El programa

```
int x;           // línea 1 ← declaración sin valor
x = 10;          // línea 2 ← asignación separada
int y = 3;       // línea 3
                // línea 4
```



```

int suma(int a, int b) {      // línea 5
    return a + b;             // línea 6
}                             // línea 7
                             // línea 8
int z = suma(x, y);           // línea 9
cout << z;                   // línea 10
float f = 3.14;               // línea 11 ← widening numérico double→float
                             // línea 12
int w = "hola";               // línea 13 ← error semántico

```

Paso 0 — Arranque

App.java:114

```
SemanticAnalyzer semantico = new SemanticAnalyzer();
```

El constructor `SemanticAnalyzer.java:18-22` inicializa:

- `tabla` → `new SymbolTable()` → crea `Scope("global", padre=null)` (`SymbolTable.java:17`)
- `errores` → lista vacía
- `advertencias` → lista vacía
- `tipoRetornoActual` → `null`

App.java:115

```
semantico.visit(arbolParseo);
```

ANTLR llama a `visitPrograma()`, que delega en `visitChildren()` e itera las sentencias de arriba hacia abajo.

Paso 1 — `int x;` (línea 1) — declaración sin valor

Método: `visitDeclaracion()`

```

[L55-57] tipo="int"  nombre="x"  token=1:4
[L59]   estaDeclaradoLocalmente("x") → false  ✓
[L64]   tieneValorInicial = false  (ctx.expresion() == null, no hay '= ...')
        → se omite el bloque de verificación de tipos
[L77-78] tabla.definir(Symbol.variable("x","int",false,1,4))

```

→ `Symbol.java:28`: crea el símbolo con `inicializado=false`.

`x` existe en la tabla pero aún no tiene valor — si se usara aquí generaría una advertencia.

```
Scope "global":  x → VARIABLE  int  NO inicializado  1:4
```

Paso 2 — `x = 10;` (línea 2) — asignación

Método: `visitAsignacion()`

```
[L84-86] nombre="x"  token=2:0
        tabla.resolver("x") [SymbolTable L42]
        → busca en scope "global" → Symbol{x, int, VARIABLE, inicializado=false}
✓
[L94]    visit(10) → visitExprEntero() [L323] → "int"
[L96-102] esCompatibleAsignacion("int","int") → true  ✓
[L104]   simbolo.setInicializado(true)  ← x ahora está inicializada
```

→ `Symbol.java:47`: el flag `inicializado` del símbolo pasa de `false` a `true`.

A partir de aquí, cualquier uso de `x` no generará advertencia.

```
Scope "global":  x → VARIABLE  int  inicializado  1:4  ← cambió de estado
```

Paso 3 — `int y = 3;` (línea 3) — declaración con valor

Método: `visitDeclaracion()`

```
[L55-57] tipo="int"  nombre="y"  token=3:4
[L59]    estaDeclaradoLocalmente("y") → false  ✓
[L64]    tieneValorInicial = true
[L67]    visit(3) → visitExprEntero() → "int"
[L68-73] esCompatibleAsignacion("int","int") → true  ✓
[L77-78] tabla.definir(Symbol.variable("y","int",true,3,4))
```

```
Scope "global":  x → VARIABLE  int  inicializado  1:4
                  y → VARIABLE  int  inicializado  3:4
```

Paso 4 — `int suma(int a, int b) { return a + b; }` (líneas 5–7)

Método: `visitDeclaracionFuncion()`

4.1 — Registrar la función

```
[L158-160] tipoRetorno="int" nombre="suma" token=5:4
[L162]      estaDeclaradoLocalmente("suma") → false ✓
[L167]      tabla.definir(Symbol.funcion("suma","int",5,4))
```

→ `Symbol.java:32:{suma, int, FUNCION, inicializado=true}`.

La función se registra en el scope actual **antes** de abrir el scope interno, para que sea visible desde fuera (y para llamadas recursivas).

4.2 — Abrir scope y registrar parámetros

```
[L168]      tabla.entrarScope("funcion_suma") → Scope("funcion_suma_1",
padre=global)
[L170-177]  tabla.definir(Symbol.parametro("a","int",5,13))
            tabla.definir(Symbol.parametro("b","int",5,20))
```

```
Scope "global" ← padre de — Scope "funcion_suma_1" ← scopeActual
                                a → PARAMETRO int 5:13
                                b → PARAMETRO int 5:20
```

4.3 — Visitar el cuerpo (return a + b)

```
[L180-181] tipoAnterior=null tipoRetornoActual="int"
[L184]      visitChildren(ctx.bloque()) → visitSentenciaReturn() [L191]
            tipoRetornoActual ≠ null → ok ✓
            visit(a + b) → visitExprAditiva() [L272]
                visit(a) → visitExprIdentificador()
                    resolver("a") → funcion_suma_1 → "int", inicializado ✓
                visit(b) → igual → "int"
                inferirAritmetico("int","int") → "int" [TypeSystem L23]
            esCompatibleAsignacion("int","int") → true ✓
```

4.4 — Cerrar scope

```
[L186] tipoRetornoActual = null (restaurado)
[L187] tabla.salirScope() → scopeActual vuelve a "global"
```

```
Scope "global":      x    → VARIABLE int 1:4
                    y    → VARIABLE int 3:4
                    suma → FUNCION  int 5:4
```

```
Scope "funcion_suma_1" [cerrado, solo en historial]:
    a    → PARAMETRO int   5:13
    b    → PARAMETRO int   5:20
```

Paso 5 — `int z = suma(x, y);` (línea 9)

Método: `visitDeclaracion()` → `visitExprLlamada()`

```
[L55-57] tipo="int" nombre="z" token=9:4
[L67]    visit(suma(x,y)) → visitExprLlamada() [L330]
        resolver("suma") → FUNCION int   ✓
        getCategoria() == FUNCION   ✓
        visit(x) → visitExprIdentificador()
            resolver("x") → Symbol{x, int, VARIABLE, inicializado=true} ✓
            (x fue inicializada en el paso 2, no genera advertencia)
        visit(y) → igual → "int"
        retorna simbolo.getTipo() → "int"
[L68-73] esCompatibleAsignacion("int","int") → true   ✓
[L77-78] tabla.definir(Symbol.variable("z","int",true,9,4))
```

```
Scope "global":    ...  z → VARIABLE int   inicializado  9:4
```

Paso 6 — `cout << z;` (línea 10)

Método: `visitSentenciaCout()`

```
[L110] visit(z) → visitExprIdentificador() [L353]
        resolver("z") → int, inicializado   ✓
        → no advertencia, retorna "int"
```

Paso 7 — `float f = 3.14;` (línea 11) — widening numérico

Método: `visitDeclaracion()`

```
[L55-57] tipo="float" nombre="f" token=11:6
[L59]    estaDeclaradoLocalmente("f") → false   ✓
[L67]    visit(3.14) → visitExprDecimal() [L324] → "double"
```

→ `SemanticAnalyzer.java:324`: los literales decimales siempre se infieren como `double`.

```
[L68-73] esCompatibleAsignacion("float","double") [TypeSystem L14]
    "float".equals("double") → false
    FLOAT.equals(hacia) && (INT o DOUBLE).equals(desde)
    → FLOAT.equals("float") ✓ y DOUBLE.equals("double") ✓
    → retorna true ✓ (widening: double cabe en float)
[L77-78] tabla.definir(Symbol.variable("f","float",true,11,6))
```

```
Scope "global": ... f → VARIABLE float inicializado 11:6
```

Paso 8 — `int w = "hola";` (línea 13) — ERROR semántico

Método: `visitDeclaracion()`

```
[L55-57] tipo="int" nombre="w" token=13:4
[L67] visit("hola") → visitExprCadena() [L326] → "string"
[L68-73] esCompatibleAsignacion("int","string") [TypeSystem L14]
    ninguna regla coincide → retorna false
    → error(token, "no se puede asignar tipo 'string' a variable de tipo
    'int'...")
```

→ `SemanticAnalyzer.java:43:`

```
errores.add(new SemanticError(13, 4, "...", Severidad.ERROR))
```

El análisis **continúa** y `w` queda registrada en la tabla para no generar errores secundarios si se usara más adelante.

Resultado final

=== FASE 3: ANÁLISIS SEMÁNTICO ===

✗ ERRORES SEMÁNTICOS (1):

[Línea 13:4] Error semántico: no se puede asignar tipo 'string' a variable de tipo 'int' en la declaración de 'w'.

TABLA DE SIMBOLOS

Scope: global [ambito raiz]

Nombre	Tipo	Categoria	Estado	Pos
x	int	VARIABLE	inicializado	1:4
y	int	VARIABLE	inicializado	3:4
suma	int	FUNCION	inicializado	5:4
z	int	VARIABLE	inicializado	9:4
f	float	VARIABLE	inicializado	11:6
w	int	VARIABLE	inicializado	13:4

Scope: funcion_suma_1 [padre: global]

Nombre	Tipo	Categoria	Estado	Pos
a	int	PARAMETRO	inicializado	5:13
b	int	PARAMETRO	inicializado	5:20

Compilacion finalizada con errores semanticos.

Diagrama del flujo completo

```

visit(programa) [SemanticAnalyzer
L48]
├─[L1] visitDeclaracion("int x") [L53]
│   ├── estaDeclaradoLocalmente("x") → false [SymbolTable L46]
│   ├── tieneValorInicial = false (sin expresión)
│   └─ definir(x, VARIABLE, int, NO inicializado) [SymbolTable L38]
├─[L2] visitAsignacion("x = 10") [L82]
│   ├── resolver("x") → Symbol{x, int, inic=false} [SymbolTable L42]
│   ├── visitExprEntero(10) → "int" [L323]
│   ├── esCompatibleAsignacion("int","int") → true [TypeSystem L17]
│   └─ simbolo.setInicializado(true) ← x ya tiene valor [Symbol L47]
├─[L3] visitDeclaracion("int y = 3") [L53]
│   └─ mismo flujo con valor → definir(y, VARIABLE, int, inicializado)
├─[L5] visitDeclaracionFuncion("int suma(int a, int b)") [L156]
│   ├── definir(suma, FUNCION, int) [SymbolTable L38]
│   ├── entrarScope("funcion_suma") → scope #1 [SymbolTable L25]
│   ├── definir(a, PARAMETRO, int) [Symbol L36]
│   ├── definir(b, PARAMETRO, int) [Symbol L36]
│   ├── tipoRetornoActual = "int" [L181]
│   ├── visitSentenciaReturn("return a+b") [L191]
│   │   └─ visitExprAditiva(a+b) [L272]
│   │       ├── visitExprIdentificador(a) → "int" [L353]
│   │       └─ visitExprIdentificador(b) → "int" [L353]

```

```

└─┬ inferirAritmetico("int","int")→"int" [TypeSystem L23]
  │└─ tipoRetornoActual = null (restaurado) [L186]
  └─ salirScope() → vuelve a "global" [SymbolTable L32]

└─[L9] visitDeclaracion("int z = suma(x,y)") [L53]
    └─ visitExprLlamada("suma") [L330]
        │└─ resolver("suma") → FUNCION int [SymbolTable L42]
        │└─ visit(x) → "int", inicializado ✓ [L353]
        │└─ visit(y) → "int", inicializado ✓ [L353]
        └─ retorna "int"

└─[L10] visitSentenciaCout("cout << z") [L108]
    └─ visitExprIdentificador(z) → "int", inic. [L353]

└─[L11] visitDeclaracion("float f = 3.14") [L53]
    │└─ visitExprDecimal(3.14) → "double" [L324]
    │└─ esCompatibleAsignacion("float","double")→true [TypeSystem L19]
    └─ definir(f, VARIABLE, float, inicializado)

└─[L13] visitDeclaracion("int w = \"hola\"") [L53]
    │└─ visitExprCadena("hola") → "string" [L326]
    │└─ esCompatibleAsignacion("int","string") → false [TypeSystem L14]
    └─ error() → errores[0] = SemanticError{13:4} [L43]

```

Resultado: errores=1 advertencias=0
 → "Compilacion finalizada con errores semanticos."